TASK 3 — STEP-BY-STEP GUIDE

DESIGN OOP MODELS · SWIFT CLASSES, UML & VALIDATION

Course: Geo Programming | Week 5 | Suggested time: 35–50 minutes | Individual work

1. Learning Outcomes

Derive domain classes from Task 1 requirements and Task 2 use cases; define properties, Swift types, initializers and methods; model associations and multiplicities; justify encapsulation, inheritance and polymorphism; and specify validation rules.

2. Scenario & Inputs

School Management System, academic year 2026–2027: students, teachers, subjects and enrollments. A teacher may teach several subjects; a subject may have several teachers. Students may enroll in several subjects. Each enrollment records an academic year and a mark (0–10). Duplicate enrollment for the same student, subject and academic year is prohibited.

Use your FR/NFR list and use-case table as evidence. Do not begin by coding all classes. Examples below illustrate the METHOD, not the complete design.

Step 1 — Identify Candidate Domain Classes (5 min)

Underline domain nouns in the specification. Keep persistent business concepts separate from SwiftUI screens and coordinating controllers.

Example 1: Student: a real-world person whose ID, name and email must be recorded.

Example 2: Subject: a course offering described by an identifier, title, credits and semester.

Example 3: StudentListView: NOT a domain class; it displays students and belongs to the UI layer.

Candidate	Domain / UI / Controller?	Reason
Student	Domain	Represents a school record
StudentListView	UI	Displays records
Teacher	Domain	Represents a school record: teacher identity and the subjects taught
Subject	Domain	Course offering with ID, title, credits and semester
Enrollment	Domain	Links one student to one subject for an academic year and holds the mark

YOUR TURN — Identify ALL remaining candidate concepts from the specification and use cases. Classify each as Domain, UI or Controller and justify the choice. Continue on extra paper if necessary.

Candidate	Layer	Justification
StudentDetailView / StudentFormView	UI	Show one student with results / collect input for add-edit; no business rules
SubjectListView, TeacherListView, EnrollmentListView	UI	Display lists (UC-08); only render data
TeacherDetailView (assign subjects), EnrollmentFormView, MarkEntryView	UI	Collect user choices and show errors; call services
ContentView (TabView)	UI	Root navigation container; must not hold data or logic
StudentController,	Controller / service	Coordinate use cases UC-02 –

EnrollmentService, TeachingAssignmentService		UC-08 and enforce cross-record rules (duplicates)
SchoolRepository (in-memory store)	Controller / service	Owns the collections of records for the in-memory lab; no persistence required
ValidationError, Describable	Domain (supporting)	Error vocabulary for failed rules / protocol for polymorphic description
Staff, AcademicYear, Mark	Not classes	Staff is an actor (no login in scope); AcademicYear and Mark are values stored as properties of Enrollment

Step 2 — Assign Responsibilities (4 min)

For each candidate class, write ONE primary responsibility. Do not place all operations in one giant class.

Example 1: Student — represent student identity and contact details.

Example 2: Subject — represent subject identity and academic attributes.

Example 3: Enrollment — represent a student taking a subject in a specific academic year, with a mark.

YOUR TURN — Determine responsibilities for ALL remaining justified domain classes or supporting abstractions. Avoid unnecessary classes; explain any excluded candidate.

Class / abstraction	Primary responsibility	Evidence (FR / UC)
Student	Represent student identity and contact details	FR-01, FR-02, FR-03, FR-08 / UC-01, 04, 05, 07
Subject	Represent subject identity and academic attributes (credits > 0)	FR-04, FR-05, FR-10 / UC-06, UC-02, UC-08
Enrollment	Represent a student taking a subject in an academic year, with a protected mark	FR-05, FR-06, FR-07, FR-11 / UC-02, UC-03
Teacher	Represent teacher identity and the set of subjects taught	FR-04, FR-10 / UC-06, UC-08
SchoolRepository	Hold the in-memory collections and answer queries (e.g. “does this enrollment exist?”)	FR-06, NFR-01 / UC-02
StudentController	Coordinate list/search, add and edit of students	FR-01, 02, 03, 09 / UC-01, 04, 05
EnrollmentService	Enforce the no-duplicate rule, apply marks, build a student’s results	FR-05 – FR-08 / UC-02, 03, 07
TeachingAssignmentService	Maintain the many-to-many teacher–subject assignment	FR-04 / UC-06
ValidationError / Validator	Name every rule failure; shared checks (non-empty, email, academic-year format)	FR-09, FR-12 / UC-04, 05
Describable (protocol)	Give each entity a readable one-line description (“understandable records”)	Task 1 “understandable records”
EXCLUDED: Staff, AcademicYear, Mark, a “Person” base class	Not needed: Staff is an actor; year and mark are plain properties; a base class is not justified (see Step 6)	—

Step 3 — Define Swift Properties, Types & Initializers (6 min)

For each domain class, identify its data, choose Swift types, define access control, and decide which fields are required when creating an instance.

Example 1: Student: id: String, name: String, email: String; init(id:name:email:).

Example 2: Subject: id: String, title: String, credits: Int, semester: Int.

Example 3: Enrollment: academicYear: String, mark: Double; also needs references/IDs to its student and subject.

Class	Properties + Swift types	Initializer parameters
Student	id: String; name: String; email: String	id, name, email
Subject	id: String; title: String; credits: Int; semester: Int	id, title, credits, semester (all required; throws on invalid)
Teacher	id: String (let); name: String; email: String; subjectIDs: Set<String> (private(set), starts empty)	id, name, email (required)
Enrollment	id: UUID (let); studentID: String (let); subjectID: String (let); academicYear: String (let); mark: Double? (private(set), nil until graded)	studentID, subjectID, academicYear (required); mark is optional and not an init parameter
ValidationError	enum conforming to Error (cases listed in Step 7)	—

YOUR TURN — Specify ALL remaining classes, properties, Swift types, access levels and initializer parameters. Mark required and optional values. Add rows as needed.

Class	Properties / Swift types / access	Initializer
Student	id: String (let); name, email: String (private(set)); access: internal; conforms to Describable, Identifiable	id, name, email – all required; init throws on blank/invalid
SchoolRepository	private(set) var students: [Student]; teachers: [Teacher]; subjects: [Subject]; enrollments: [Enrollment]; final class, ObservableObject (or @Observable)	init(students:teachers:subjects:enrollments:) – optional, default sample data
StudentController / EnrollmentService / TeachingAssignmentService	private let repository: SchoolRepository (dependency injection)	repository (required)
Validator	enum with static funcs: nonEmpty(_:), isValidEmail(_:), isValidAcademicYear(_:), isValidMark(_:)	none (namespace, never instantiated)
Describable	protocol with func describe() → String	—

Step 4 — Identify Meaningful Methods & Validation (5 min)

Use verbs from use cases. A method should perform domain behavior or protect an invariant; do not add trivial getters/setters without a reason.

Example 1: Student.updateEmail(_:) — accept a nonempty, valid-looking email or report an error.

Example 2: Enrollment.updateMark(_:) — accept only a score in 0...10.

Example 3: Subject.updateCredits(_:) — reject nonpositive credit values (proposed design rule; label as an assumption).

Class	Method signature (proposed)	Validation / result
Enrollment	func updateMark(_ value: Double) throws	0...10 or error
Student	func updateEmail(_ value: String) throws	valid email or error
Subject	func updateCredits(_ value: Int) throws	value > 0 else invalidCredits (assumption)
Teacher	func assign(subjectID: String) throws	throws duplicateAssignment if already in subjectIDs (assumption)
Student	func updateName(_ value: String) throws	nonempty else emptyField; previous name kept on error

YOUR TURN — Derive ALL remaining meaningful methods and validation rules from your use cases. Explain expected outcomes and label proposed rules as assumptions.

Class	Method / signature	Validation / expected result
Enrollment	init(studentID:subjectID:academicYear:) throws	all nonempty; year matches YYYY–YYYY (consecutive); mark starts nil
EnrollmentService	func enroll(studentID:subjectID:year:) throws → Enrollment	unknown IDs → notFound; same student+subject+year → duplicateEnrollment; list unchanged on error
EnrollmentService	func updateMark(enrollmentID: UUID, value: Double) throws	delegates to Enrollment.updateMark; 0...10 else markOutOfRange
EnrollmentService	func results(forStudent id: String) → [(subject: Subject, year: String, mark: Double?)]	returns every enrollment of the student; empty array if none
StudentController	func addStudent(id:name:email:) throws / updateStudent(id:name:email:) throws	blank fields → emptyField; bad email → invalidEmail; existing ID → duplicateID (assumption)
StudentController	func loadStudents(matching query: String) → [Student]	empty query returns all; no match returns []
TeachingAssignmentService	func assign(subjectID:toTeacher:) throws; teachers(forSubject:) → [Teacher]	many-to-many allowed; duplicate assignment rejected
Describable (all entities)	func describe() → String	e.g. “ST001 – Nguyen Van An”; never returns an empty string

Step 5 — Draw UML Associations & Multiplicities (5 min)

Represent relationships with lines and multiplicities (1, 0..*, etc.). Do not confuse inheritance with association.

Example 1: Student (1) — (0..*) Enrollment: one student can have many enrollment records.

Example 2: Subject (1) — (0..*) Enrollment: one subject can appear in many enrollments.

Example 3: Teacher (0..*) — (0..*) Subject: teaching assignments are many-to-many.

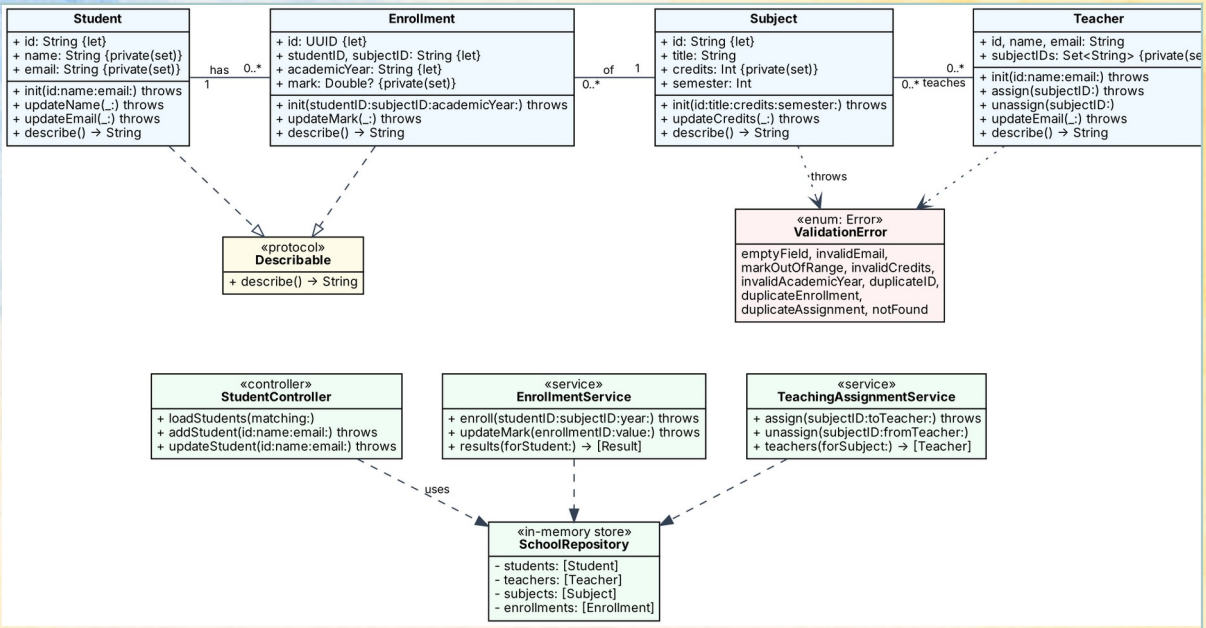
Simple UML-style notation: [Student] 1 ——— 0..* [Enrollment] 0..* ——— 1 [Subject]

Class A	Multiplicity	Class B	Explanation
Student	1 → 0..*	Enrollment	Student has enrollments
Teacher	0..* ↔ 0..*	Subject	Teachers teach subjects
Subject	1 → 0..*	Enrollment	A subject appears in many enrollments
Enrollment	{unique: studentID + subjectID + academicYear}	—	Constraint on the enrollment collection: no duplicates

YOUR TURN — Identify ALL justified associations and multiplicities. Complete the relationship worksheet and draw your full UML-style class diagram in the drawing area.

Class A	Multiplicity	Class B	Rationale
Student	1 — 0..*	Enrollment	A student can take many subjects; enrollment stores studentID
Subject	1 — 0..*	Enrollment	Many students can take the same subject; enrollment stores subjectID
Teacher	0..* — 0..*	Subject	Teaching assignment is many-to-many; stored once in Teacher.subjectIDs (a subject's teachers are queried)
SchoolRepository	1 ◆— 0..*	Student, Teacher, Subject, Enrollment	Composition: the store owns the records (arrays)
StudentController, EnrollmentService, TeachingAssignmentService	* ⇢ 1	SchoolRepository	Dependency: services read/modify the store; views never do
Student, Teacher, Subject, Enrollment	realization ⇢▷	Describable	Protocol conformance (not inheritance)

UML CLASS DIAGRAM — Draw your complete model here (or attach a separate diagram):



Notes: Teacher and Subject also conform to Describable (arrows omitted for clarity). SchoolRepository owns the four record arrays (composition). All throwing methods use ValidationError. Views (UI layer) are intentionally not part of this domain diagram.

Step 6 — Explain OOP Principles (7 min)

Show HOW the design applies a principle. Inheritance and polymorphism must be justified by real behavior; a protocol or composition may be a better choice.

Example 1: Encapsulation: private(set) var mark: Double; a validated updateMark(_:) method prevents uncontrolled writes.

Example 2: Inheritance (optional): a Person superclass could share name/email between Student and Teacher, but only if common identity/behavior justifies it.

Example 3: Polymorphism: a Describable protocol with describe() can be implemented differently by Student and Teacher; call it through a Describable reference.

Swift starter syntax (illustrative only):

```
protocol Describable { func describe() -> String }
class ExampleEntity {
    private(set) var id: String
    init(id: String) { self.id = id }
    func validate() -> Bool { !id.isEmpty }
}
```

YOUR TURN — Examine ALL relevant OOP decisions: encapsulation, abstraction, inheritance, overriding, protocols, polymorphism and composition. Explain where each applies or why it is unnecessary.

OOP principle / class	Design decision / Swift feature	Why appropriate?
Encapsulation – Enrollment	private(set) var mark: Double?; changed only by updateMark(_:) throws	An invalid mark can never be stored and the previous mark survives a failed update
Encapsulation – Student, Teacher, Subject, SchoolRepository	let id; private(set) name/email/credits/subjectIDs and record arrays	Identity is fixed; every change passes a validating method or service, so views cannot append duplicates
Abstraction	Services expose intent (enroll, assign, results) not storage	Views stay simple; the in-memory store could later be replaced without changing the UI
Inheritance (Person base class)	NOT used; final class for all entities	Student and Teacher share only id/name/email and no behavior; a hierarchy would be forced.

		Duplication is tiny
Overriding	Not applicable	No subclasses exist, so nothing is overridden; final also blocks accidental subclassing
Protocol + polymorphism – Describable	describe() → String implemented by Student, Teacher, Subject, Enrollment; called through a [Describable] reference	Each type formats its own fields; one generic row/summary component works for all
Composition	Enrollment holds studentID/subjectID; services hold a SchoolRepository	Avoids reference cycles and a second copy of the data (single source of truth)
Error handling with throws	enum ValidationError: Error	Views translate each case into the message shown in the prototype (S11, S12, S13)

Step 7 — Protect Business Rules (5 min)

Turn each business rule into a testable invariant. Decide which model or service checks it, and how errors are reported to the controller/UI.

Example 1: Invalid mark: updateMark(11) must fail and preserve the previous mark.

Example 2: Duplicate enrollment: adding the same student + subject + academic year twice must be rejected.

Example 3: Valid boundary: marks 0 and 10 must be accepted.

Rule / case	Owner of validation	Expected behavior
mark = 11	Enrollment / enrollment service	Reject; keep previous value
same student + subject + year	Enrollment manager / repository	Reject duplicate
mark = 0	Enrollment.updateMark	Accept (lower boundary)
mark = 10	Enrollment.updateMark	Accept (upper boundary)
mark = −0.1	Enrollment.updateMark	Reject markOutOfRange; keep previous value

YOUR TURN — Find ALL meaningful valid, invalid, duplicate and boundary cases implied by the specification. State who enforces each rule and what happens when validation fails.

Rule / test case	Validation owner	Expected behavior
mark = 10.1, or NaN	Enrollment.updateMark	Reject markOutOfRange; keep previous value
mark text blank or “abc”	Mark entry view (parse) – updateMark is never called	Show “Mark must be 0–10”; saved mark unchanged
same student + subject + year, twice	EnrollmentService.enroll (checks repository)	Reject duplicateEnrollment; enrollment count unchanged
same student + subject, different year	EnrollmentService.enroll	Accept (not a duplicate)
same student + different subject, same year; or different student + same subject, same year	EnrollmentService.enroll	Accept
unknown studentID or subjectID	EnrollmentService.enroll	Reject notFound (assumption: referential integrity)
blank ID, name or email	Student.init / Teacher.init (Validator.nonEmpty)	Reject emptyField; no object created
email “abc”, “a@b”, “a b@c.com” vs “an@school.edu”	Validator.isValidEmail	First three: reject invalidEmail; last: accept
credits 0 or −1 vs 1	Subject.init / updateCredits	Reject invalidCredits (assumption); 1 accepted
academic year “2026–2027” vs “2026–2028”, “26–27”, “”	Validator.isValidAcademicYear	First: accept; others: reject invalidAcademicYear (format is a design assumption)
new student with an existing ID	StudentController.addStudent	Reject duplicateID (assumption); existing record unchanged
assign the same subject to the same teacher twice	Teacher.assign / TeachingAssignmentService	Reject duplicateAssignment (assumption); no change
one subject to two teachers; one teacher with several subjects	TeachingAssignmentService.assign	Accept (many-to-many stated in the specification)

Step 8 — Review, Trace & Submit (3 min)

Trace your classes and operations back to Task 1 FRs and Task 2 use cases.

Example 1: Student + student listing → FR for browsing students.

Example 2: Enrollment.updateMark → use case for entering/updating marks.

Example 3: Duplicate-enrollment check → business rule and alternate/error flow.

Requirement / use case	Model or method	Evidence / test

FR-01 / UC-01	Student; StudentController.loadStudents(matching:)	Load 5 sample students → IDs and names returned; query "zzz" → []
FR-02, FR-09 / UC-04	StudentController.addStudent → Student.init (throws)	Blank ID → emptyField; valid input → student count +1
FR-03 / UC-05	Student.updateName/ updateEmail; StudentController.updateStudent	New email "x@school.edu" stored; invalid email → old value kept
FR-05, FR-06 / UC-02	EnrollmentService.enroll + duplicate check in SchoolRepository	Enroll same triple twice → second throws duplicateEnrollment; count unchanged

YOUR TURN — Trace ALL remaining classes, methods and validation decisions to Task 1 requirements and Task 2 use cases. Identify any uncovered requirement.

FR / UC ID	Model / method / rule	Evidence / test
FR-07 / UC-03	Enrollment.updateMark via EnrollmentService.updateMark	Marks 0 and 10 accepted; 11 and −0.1 rejected with old mark preserved
FR-04 / UC-06	Teacher.subjectIDs; TeachingAssignmentService.assign	Subject given to two teachers → both list it; duplicate assignment rejected
FR-08 / UC-07	EnrollmentService.results(forStudent:)	Student with 2 enrollments → 2 rows with year and mark (nil shown as "—")
FR-10, FR-11 / UC-08	SchoolRepository queries; Describable.describe()	Lists return all subjects / teachers / enrollments (proposed FRs)
FR-12	Validator.isValidEmail / isValidAcademicYear; Subject credits check	Boundary tests from the Step 7 table
NFR-01 (performance)	Arrays + linear filters in SchoolRepository	Generate 100 enrollments; results(forStudent:) completes well under the 2 s criterion
NFR-04 (robustness)	Every failure path uses throws, never a force-unwrap or trap	Duplicate and out-of-range tests produce zero crashes
UNCOVERED	Add/edit of teachers and subjects; NFR-02 and NFR-03 (layout, message timing)	Teachers/subjects are sample data (assumption). NFR-02/03 are UI qualities and belong to Task 4

3. Required Student Deliverables

☐ Candidate-class classification with responsibilities and rationale.

☐ Completed model design worksheet: attributes/types, access control, initializers, methods and validation.

☐ One UML-style class diagram showing ALL justified associations and multiplicities.

☐ OOP explanation: encapsulation plus a justified inheritance/protocol/composition decision; show polymorphism if appropriate.

☐ Business-rule table with duplicate-enrollment and mark-range checks, including boundary cases.

☐ Traceability table linking model decisions to Task 1 FRs and Task 2 use cases.

Save as Task3_OOP_Models_StudentID.pdf and include it in Design.pdf for Lab 01. Submit your own design; the worked examples are not a complete solution.

4. Self-Assessment Checklist

☐ Every domain class has one clear responsibility.

☐ Swift types and initializers are specified for every class.

☐ UI views and controllers are not mistaken for domain entities.

☐ UML relationships have correct multiplicities.

☐ No unnecessary inheritance hierarchy has been forced.

☐ Invalid marks and duplicate enrollments cannot silently alter data.

☐ Every key model behavior traces to a requirement or use case.

5. Common Mistakes

Common mistake	Better approach
Put all data and logic in ContentView	Separate domain models, views and coordination
Use inheritance only because fields repeat	Prefer composition/protocols unless subtype behavior is meaningful
Expose mark as freely writable	Protect it with validated mutation
Draw UML with no multiplicities	Add 1, 0..*, etc., to every association
Invent new business rules as facts	Label them as design assumptions

Reflection (3–5 sentences): Which relationship or validation rule most influenced your model design? What trade-off did you make between inheritance and composition?

The many-to-many teacher–subject relationship and the no-duplicate-enrollment rule shaped the model most. Because an enrollment is a record of its own (student + subject + year + mark), I made Enrollment a separate class with IDs instead of letting Student hold subject objects, and put the duplicate check in EnrollmentService, which can see all enrollments. I stored teaching assignments once, in Teacher.subjectIDs as a Set, which also blocks duplicate assignments. For inheritance versus composition I rejected a Person superclass because Student and Teacher share only three fields and no behavior; a Describable protocol and a shared Validator give reuse without a forced hierarchy.